Amazon Verified Permissions Security

IDENTITY

Amazon Verified Permissions is a fully managed, fine-grained authorization service for custom applications. It uses the Cedar policy language to define permissions and make authorization decisions. Applications call IsAuthorized to evaluate Cedar policies stored in a policy store.

MEDIUM	10+	Cedar	RBAC + ABAC
Risk Level	Attack Vectors	Policy Language	Auth Model

📋 Service Overview

Policy Stores & Cedar Policies

Policy stores hold Cedar policies, schemas, and identity sources. Cedar uses explicit permit and forbid effects with default-deny. Any forbid match results in Deny; else any permit match results in Allow; otherwise Deny.

Attack note: An overly broad permit(principal, action, resource) with no conditions allows any authenticated user to perform any action on any resource -- the Cedar equivalent of IAM Allow *.

Identity Sources & Integration

Identity sources connect Cognito user pools or OIDC providers to policy stores. IsAuthorizedWithToken accepts ID/access tokens directly and maps claims to Cedar entities.

Attack note: If an attacker can inject Cedar policies via CreatePolicy, they can create permit(principal, action, resource) granting themselves full access to every application resource.

Security Risk Assessment

6.0

Risk Score

Verified Permissions is an application-layer authorization service. It does not directly control AWS resources. However, misconfigurations in Cedar policies, schemas, or identity source mappings can lead to authorization bypass in the applications that rely on it.

⚔️ Attack Vectors

Cedar Policy Logic Attacks

- Exploit gaps between permit and missing forbid policies
- Abuse wildcard scopes in Cedar policies
- Principal identifier reuse (human-readable vs UUID)
- Overly broad policy templates when instantiated
- Missing unless conditions on permit policies

Infrastructure Attacks

- Enumerate policies and schemas to map auth model
- Tamper with identity source token claims
- Exploit disabled schema validation (mode=OFF)
- Inject policies via verifiedpermissions:CreatePolicy
- Bypass Verified Permissions if app skips IsAuthorized

⚠️ Misconfigurations

Cedar Policy Issues

- Schema validation set to OFF in production
- Overly broad permit with no constraints
- Human-readable identifiers (User::"jane") instead of UUIDs
- Missing forbid policies for sensitive actions
- Templates with unscoped placeholders

Infrastructure Issues

- IAM granting verifiedpermissions:* to app roles
- No separation between read (IsAuthorized) and write (CreatePolicy)
- Identity source without client ID restrictions
- CloudTrail data events not enabled for IsAuthorized
- No monitoring on policy store modifications

🔍 Enumeration

List All Policy Stores

```
aws verifiedpermissions
list-policy-stores
```

List All Policies in a Store

```
aws verifiedpermissions
list-policies \
  --policy-store-id
PSEXAMPLEabcdefg111111
```

Get a Specific Policy

```
aws verifiedpermissions
get-policy \
  --policy-store-id
PSEXAMPLEabcdefg111111 \
  --policy-id
SPEXAMPLEabcdefg111111
```

Get the Schema

```
aws verifiedpermissions
get-schema \
  --policy-store-id
PSEXAMPLEabcdefg111111
```

List Identity Sources

```
aws verifiedpermissions
list-identity-sources \
  --policy-store-id
PSEXAMPLEabcdefg111111
```

📈 Privilege Escalation

Cedar Policy Injection

- CreatePolicy to inject blanket permit(principal, action, resource)
- UpdatePolicy to widen scope of existing policies
- PutSchema to add new entity types/actions
- Delete forbid policies to weaken controls
- Create template-linked policies with broad templates

Application-Level Escalation

- Principal identifier reuse (inherit former user permissions)
- Abuse Cognito group claims mapped to Cedar entities
- Exploit missing IsAuthorized checks on endpoints
- Probe authorization decisions via IsAuthorized API
- Analyze policies offline to find authorization gaps

Key insight: Enumerate the full policy set with list-policies and get-policy, then analyze Cedar logic offline to find gaps -- actions with no forbid and overly broad permits.

🔗 Persistence

Policy Manipulation

- Inject persistent permit policies in policy store
- Delete forbid policies that restrict access
- Modify schema to add attacker-controlled entity types
- Create template-linked policies from broad templates
- Register identity sources with attacker OIDC provider

Identity-Level

- Reuse identifiers of deleted principals
- Add attacker to Cognito groups mapped to Cedar permissions
- Maintain access via identity source token issuance
- Create additional policy stores for parallel access
- Modify validation to OFF to accept malformed policies

🛡️ Detection

Management Events (Default)

- CreatePolicy - new policy added
- DeletePolicy - policy removed
- UpdatePolicy - policy modified
- PutSchema - schema updated
- CreateIdentitySource - identity source added

Data Events (Must Enable)

- IsAuthorized - authorization decision requested
- IsAuthorizedWithToken - token-based auth decision
- BatchIsAuthorized - batch authorization decisions
- Spike in DENY results (authorization probing)
- Unusual CreatePolicy from non-pipeline principals

💻 Exploitation Commands

Inject Blanket Permit Policy

```
aws verifiedpermissions create-policy \
  --policy-store-id PSEXAMPLEabcdefg111111 \
  --definition '{"static":{"statement":"permit(principal, action,
resource);","description":"backdoor"}}'
```

Delete a Forbid Policy

```
aws verifiedpermissions delete-policy \
  --policy-store-id PSEXAMPLEabcdefg111111 \
  --policy-id SPEXAMPLEforbid111111
```

Test Authorization Decision

```
aws verifiedpermissions is-authorized \
  --policy-store-id PSEXAMPLEabcdefg111111 \
  --principal entityType=User,entityId=attacker \
  --action actionType=Action,actionId=admin \
  --resource entityType=Resource,entityId=sensitive
```

Disable Schema Validation

```
aws verifiedpermissions update-policy-store \
  --policy-store-id PSEXAMPLEabcdefg111111 \
  --validation-settings "mode=OFF"
```

📜 Policy Examples

✗ Dangerous - Full Verified Permissions Access

```
{
  "Effect": "Allow",
  "Action": "verifiedpermissions:*",
  "Resource": "*"
}
```

Grants ability to create, modify, and delete policies, schemas, and identity sources -- can rewrite entire auth model

✓ Secure - Application Auth Only

```
{
  "Effect": "Allow",
  "Action": [
    "verifiedpermissions:IsAuthorized",
    "verifiedpermissions:IsAuthorizedWithToken",
    "verifiedpermissions:BatchIsAuthorized"
  ],
  "Resource": "arn:aws:verifiedpermissions::*:policy-store/
PSEXAMPLEabcdefg111111"
}
```

Application role can only make authorization decisions. Cannot modify policies, schemas, or identity sources.

✗ Dangerous - Cedar Blanket Permit

```
permit(principal, action, resource);
```

Allows any principal to perform any action on any resource. The Cedar equivalent of IAM Allow *.

Defense Recommendations

Enable STRICT Schema Validation

Always enable STRICT validation in production to ensure policies are validated against the schema.

```
aws verifiedpermissions update-policy-store \
  --policy-store-id PSEXAMPLEabcdefg111111 \
  --validation-settings "mode=STRICT"
```

Separate Auth vs Admin IAM Permissions

Grant app roles only IsAuthorized. Reserve CreatePolicy, UpdatePolicy, PutSchema for deployment pipelines.

Enable CloudTrail Data Events

IsAuthorized and IsAuthorizedWithToken are data events not logged by default. Enable explicitly.

Use UUIDs for All Entity Identifiers

Never use human-readable names that can be reassigned. Use UUIDs for all principals and resources.

Implement Forbid Policies for Sensitive Actions

Do not rely solely on absence of permits. Create explicit forbid policies for sensitive actions.

Manage Policies as Code

Store Cedar policies in version control. Use CI/CD pipelines with code review for all policy changes.

Amazon Verified Permissions Security Security Card

Always obtain proper authorization before testing